

MIPS Pipeline Simulator

CS3339 · Spring 2026 · 5-Stage Pipelined CPU

Sarbocha Pandey · Santiago Bolanes · Antonio Perez · Tyerrance Johnson



Due: May 4, 2026

What Did We Build?

We built a program that pretends to be a real CPU chip.



Input

You give it a text file of MIPS assembly instructions.



Process

It runs those instructions one cycle at a time through 5 stages; just like a real CPU.



Output

It prints what every register and memory slot contains at the end.



Debug

It can also print exactly what is happening inside every pipeline stage each clock cycle.

Why Does Pipelining Matter?

Think of a laundry line:

Without a pipeline: wash one load completely, THEN start the next. With a pipeline: while one load dries, the next is washing — multiple loads move at the same time!

Faster throughput

A new instruction starts every single clock cycle instead of waiting for the previous one to fully finish.

Parallel work

While one instruction is in the Execute stage, the next is being Decoded, and another is being Fetched; all at once.

Real-world concept

Every modern CPU (Intel, AMD, Apple Silicon) uses pipelining. This project shows you exactly how it works from the inside.

Program Structure

Everything lives in one C++ file. It has four main building blocks:

1

Instruction Struct

A box that holds one instruction's info (what operation, which registers, what number).

2

Memory Class

A list of 1024 slots that the CPU can read from or write to — like RAM.

3

Pipeline Registers

Four 'handoff trays' that pass data from one stage to the next each cycle.

4

CPU Class

The main engine. Runs the 5-stage loop every cycle and updates everything.

Instruction Struct – What It Stores

Every instruction gets packed into this struct before it enters the pipeline:

Code

```
struct Instruction {  
    string opcode;  
    int rs = 0;  
    int rt = 0;  
    int rd = 0;  
    int immediate = 0;  
    int address = 0;  
    string raw;  
};
```

opcode	The instruction name e.g. "ADD", "LW", "BEQ"
rs	Source register 1 e.g. register #3 = R3
rt	Source register 2 or destination for I-type
rd	Destination register where the result is saved (R-type)
immediate	A constant number baked into the code e.g. ADDI R1, R0, 10 → 10
address	Jump target address used only by the J instruction
raw	The original text line e.g. "ADD 3, 1, 2"

Pipeline Registers – The Handoff Trays

After each stage finishes, it drops its results into a register so the next stage can pick them up.

Pipeline Register Structs

```
// Between IF and ID
struct
IF_ID {
    Instruction
    instr;
    bool
    valid = false; };

// Between ID and EX

struct
ID_EX {
    Instruction
    instr;
    int
    readData1, readData2;
    bool
    valid = false; };

// Between EX and MEM

struct
EX_MEM {
    int
    aluResult, writeData;
    bool
    valid = false; };

// Between MEM and WB

struct
MEM_WB {
    int
    memData, aluResult;
    bool
    valid = false; };
```

The valid flag tells each stage whether the tray actually has something in it.

IF/ID

Passes the fetched instruction from Fetch to Decode.

ID/EX

Passes decoded register values into the Execute stage.

EX/MEM

Passes the ALU result and write data into Memory.

MEM/WB

Passes loaded memory data or ALU result to Write-Back.

CPU Execution – The Main Loop

Every single clock cycle, the CPU runs all 5 stages in this exact order:

```
while (pipeline not empty) {  
    WB();  
    MEM();  
    EX();  
    ID();  
    IF();  
}
```

💡 Why WB first, IF last?

Running stages back-to-front each cycle means each stage reads data from the previous cycle's register before it gets overwritten by this cycle. It's the simplest way to keep all 5 stages moving at once without bugs.

IF – Fetch

Grab the next instruction from the program list. Advance the PC counter.

ID – Decode

Figure out what the instruction wants. Read the two source registers.

EX – Execute

Do the math. Add, subtract, compute the memory address, check branch.

MEM – Memory

Load a value from memory (LW) or save a value to memory (SW).

WB – Write-Back

Write the final result back into the destination register.

Binary Instruction Encoding

Every instruction is also printed as its 32-bit binary form — exactly as a real CPU would see it.

Format	Opcode	rs	rt	Last bits
R-Type (ADD, SUB, MUL...)	000000	5 bits	5 bits	rd(5) + shamt(5) + funct(6)
I-Type (ADDI, LW, SW, BEQ)	varies	5 bits	5 bits	16-bit immediate / offset
J-Type (J)	000010	—	—	26-bit jump address

Example → **ADDI 1, 0, 10** ("Set register 1 to the number 10")

001000 00000 00001 00000000000001010

opcode rs=0 rt=1 immediate value = 10 in binary

Control Signals – Telling Stages What To Do

Based on the opcode, the CPU flips these 5 switches to control what happens:

RegWrite	Save result to a register? Used by: ADD SUB MUL AND OR ADDI LW
MemRead	Read a value FROM memory? Used by: LW
MemWrite	Write a value INTO memory? Used by: SW
Branch	Jump to a new line if two registers match? Used by: BEQ
Jump	Jump to a hard-coded address? Used by: J

Hazard Handling – Keeping Instructions Safe

⚠️ What is a data hazard?

If instruction B needs a value that instruction A is still computing, B might read the old (wrong) value. That's a hazard.

Our Approach

- No automatic hazard detection
- No forwarding/bypassing
- Programmer adds NOP instructions
- 2 NOPs needed after each write
- Branch/Jump flush IF and ID stages

Future Plan

- Automatic hazard detection unit
- Forwarding paths (bypass stalls)
- Auto-insert NOPs / stall cycles
- Branch predictor
- GUI waveform visualizer

Sample Input Program

This is the assembly file we feed into the simulator — each line is one instruction:

sample.asm

```
ADDI 1, 0, 10 # R1 = 10
NOP
NOP
ADDI 2, 0, 5 # R2 = 5
NOP
NOP
ADD 3, 1, 2 # R3 = R1+R2 = 15
NOP
NOP
SUB 4, 1, 2 # R4 = R1-R2 = 5
NOP
NOP
MUL 5, 1, 2 # R5 = R1×R2 = 50
```



ADDI — Add Immediate

Sets a register to a fixed number. ADDI 1, 0, 10 means: $R1 = R0 + 10 = 10$.



NOP — No Operation

Does nothing for one cycle. We add 2 NOPs after each instruction so results are ready before the next instruction reads them.



ADD / SUB / MUL

Standard math on two registers. Result goes into the third register listed.

Sample Output — Final Register State

After the program finishes, the simulator prints every register's value:

Final Register State

```
R0      = 0  # Always 0 – the zero register
R1      = 10 # ADDI 1, 0, 10
R2      = 5  # ADDI 2, 0, 5
R3      = 15 # ADD 3, 1, 2 → 10 + 5
R4      = 5  # SUB 4, 1, 2 → 10 - 5
R5      = 50 # MUL 5, 1, 2 → 10 × 5
R6..R31 = 0  # Untouched
```



Debug Mode

Each cycle also prints:

- Cycle number
- IF/ID instruction + binary
- ID/EX values
- EX/MEM ALU result
- MEM/WB data
- Control signals
- Registers R0–R7

How the Code Works Together

From file to final answer — here's the journey every instruction takes:

1



Parser reads the .asm file

Opens "sample.asm" line by line and fills a vector of Instruction objects. Each word on the line becomes a field in the struct.

```
parseFile("sample.asm") → vector<Instruction>
```

2



CPU loops over every cycle

While there are still instructions in the pipeline, the CPU calls all 5 stage functions once per cycle.

```
while (PC < size || pipeline has valid stages)
```

3



Pipeline registers pass data

Each stage reads from the register BEFORE it and writes to the register AFTER it. Data flows forward one stage per cycle.

```
id_ex.readData1 = registers[if_id.instr.rs]
```

4



WB writes results back

The final stage takes the computed value and stores it in the register file or memory. This is what makes the program actually do something.

```
registers[instr.rd] = mem_wb.aluResult
```

Challenges We Faced

Building a pipeline is trickier than it sounds. Here's what tripped us up:

1



Getting the stage order right

Problem: If you run IF before WB in the same cycle, WB overwrites pipeline registers before earlier stages read them.

Fix: We run WB → MEM → EX → ID → IF every cycle so each stage reads last cycle's data.

2



Getting branch (BEQ) to work

Problem: When a branch is taken, instructions that already entered IF and ID need to be cancelled — otherwise wrong instructions run.

Fix: When BEQ triggers, we set `if_id.valid = false` and `id_ex.valid = false` to flush the two bad instructions.

3



Handling data hazards without forwarding

Problem: If `ADD R3, R1, R2` runs right after `ADDI R1, 0, 10`, R1 hasn't been written back yet — ADD reads the old value.

Fix: We add 2 NOP instructions as spacers so WB has time to finish writing before the next read happens.

Future Improvements

If we had more time, here's what we'd add next:

Forwarding / Bypassing

Route the result of EX or MEM directly back to the input of EX — no NOPs needed. The most common real CPU trick.

Hazard Detection Unit

Hardware logic that automatically detects when a stall is needed and pauses the pipeline for one cycle.

Branch Prediction

Guess which way a branch will go so the CPU doesn't have to flush and wait every time.

Performance Metrics (CPI)

Count total cycles and instructions to calculate CPI (cycles per instruction) — a real benchmark number.

GUI Visualizer

A graphical view showing instructions moving through each stage in real time — makes the pipeline obvious to watch.

Thank You

Questions & Discussion

What language?

C++ (single file)

How many stages?

5 IF ID EX MEM WB

Hazard handling?

Manual NOPs

Debug mode?

**Yes – per-cycle
output**

Sarbocha Pandey

Santiago Bolanes

Antonio Perez

Tyerrance Johnson